

Project Report: Stock Trader AI

Authors

Eric Cao, 28438302, caoe5@uci.edu

Shailey Joseph, 39473623, shaileyj@uci.edu

1. Introduction and Problem Statement

In this project, we build a reinforcement learning model that learns profitable stock trading strategies from real market data and social media sentiment. Given a year of hourly price data and tweets for 13 stocks, our goal is to build an agent that observes the market, adjusts its portfolio accordingly, and ultimately maximizes returns while minimizing risk. To exploit the relationships that many stocks share with each other, we first cluster the stocks based on their price trajectories.

After clustering, we train a Proximal Policy Optimization (PPO) agent for each cluster. Each agent specializes in a group of related stocks, and learns how to reallocate their cluster-specific portfolios over time, using rewards derived from changes in portfolio value and risk of their actions. Evaluation of our model is done by comparing each agent's performance during the testing period against a buy-and-hold strategy. Our results indicate that our model outperforms the baseline strategy by 4.6% during the six-week testing period, demonstrating the potential for the use of reinforcement learning in stock trading.

2. Related Work

The application of reinforcement learning to stock trading has been widely explored. *Reinforcement Learning Framework for Quantitative Trading* (Gill and Yasin, 2024) at <https://arxiv.org/html/2411.07585v1> explores modern RL approaches to trading and discusses the numerous design choices in creating an RL model. They discuss how the use of various technical indicators in state representation, continuous and discrete action spaces, and the different reward functions affect performance. Their study evaluates three RL algorithms (A2C, PPO, and DQN) and explores the strengths and weaknesses of each algorithm. Our project builds directly on this by adopting PPO as the core RL algorithm and using public sentiment via tweets as a technical indicator in the state.

Prior students have also explored the use of reinforcement learning in stock trading. For instance, a project from last year attempted to train an RL model using the soft actor-critic (SAC) algorithm, combining market data with sentiment analysis of news articles. Our project explores a different direction by focusing on social media sentiment, specifically tweets. Since tweets can be made by anyone, this provides a more informal source of information that is more reflective of public opinion. In addition, our project also clusters the stocks based on their price trajectories, allowing each agent to specialize in a subset of stocks that behave similarly to each other.

3. Data Sets

The first dataset we use is <https://www.kaggle.com/datasets/olegshpagin/usa-stocks-prices-ohlcv>. This dataset contains the hourly stock prices of 514 US stocks from January 1998 to March 2024. Each entry in this dataset contains a stock's price at the start of the hour, the highest price reached during the hour, the lowest price reached during the hour, the price at the end of the hour, and the volume traded during the hour.

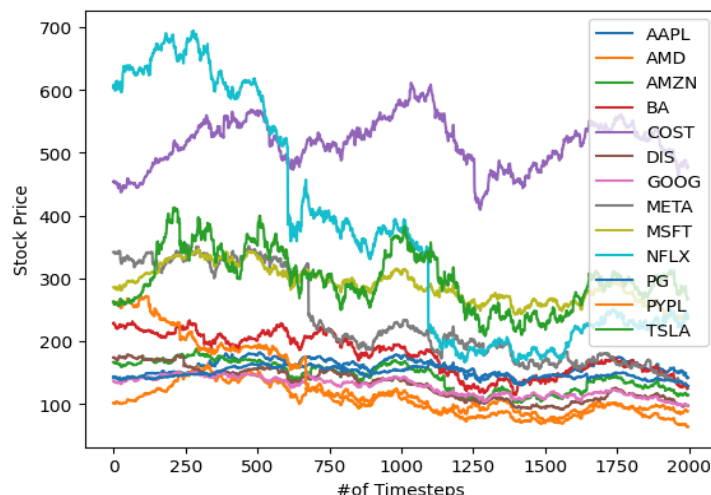


Figure 1: Hourly movement of some stocks over span of a year, as obtained from the stocks dataset

We also use the dataset found at <https://www.kaggle.com/datasets/equinxx/stock-tweets-for-sentiment-analysis-and-prediction> for sentiment analysis. This dataset contains about 65k tweets mentioning 25 companies from September 2021 to September 2022. Each entry in this dataset contains the tweet's timestamp, text, and company mentioned. Unfortunately, this dataset is slightly flawed. For one, TSLA is disproportionately represented in the dataset, as 46% of tweets are about TSLA. Additionally, many of the stocks have sparse tweets, with NOC having only 31 tweets for an entire year. We decided to cut all stocks with less than 365 tweets so there would still be about one tweet per day, even if this data is still somewhat incomplete since we're actually using hourly data. This was so that there would be enough stocks for us to try our clustering methodology. We also cut TSM, since upon closer inspection, the majority of tweets were not about the stock, but about the esports organization of the same name. In the end, we only used 13 stocks: AAPL, AMD, AMZN, BA, COST, DIS, GOOG, META, MSFT, NFLX, PG, PYPL, and TSLA.

Stock Name		DIS	635
TSLA	37422	BA	399
TSM	11034	COST	393
AAPL	5056	INTC	315
AMZN	4089	KO	310
MSFT	4089	CRM	233
PG	4089	XPEV	225
NIO	3021	ENPH	216
META	2751	ZS	193
AMD	2227	VZ	123
NFLX	1727	BX	50
GOOG	1291	F	31
PYPL	843	NOC	31

Figure 2: List of stocks in the sentiment dataset, with number of tweets about each stock

We use this dataset <https://fred.stlouisfed.org/series/DTB3> as the risk-free rate. Each entry represents the yield of the 3-month treasury bill during each day, and we used it as a reference of the risk-free rate for calculating the Sharpe ratio.

4. Description of Technical Approach

We added a sentiment column to our stock data based on the sentiment of tweets written at each hour. In order to do this, we ran an existing sentiment analysis model from huggingface that would return a value between 0 and 4, with 0 being very negative and 4 being very positive. Since our data is sparse and not every hour for a specific tweet is guaranteed to even have tweets, we subtracted 2 from this output so that a negative number always signals a negative sentiment, and a positive number always indicates a positive sentiment. A sentiment of 0 would either indicate that sentiment is neutral/balanced or indicate that there are no tweets in our dataset written about that stock in that particular hour. In order to combine the sentiments with the stock data, we simply stripped the tweet timestamp of the minutes and seconds to get the hour it was written in and used binary search on the stock data, which was already in order, to find the row corresponding to the correct hour. Then, we added the sentiment for that row to a new sentiment column.

Prior to creating and running the model, we also cluster the stocks based on their price trajectories. Our intuition behind this is that stocks within the same cluster will have similar changes in price, so the optimal move for one stock in a cluster will be similar to the optimal move for the other stocks in the same cluster. This will make it easier for a model to learn a cluster's behavior. To cluster the stocks, we decided to use hierarchical agglomerative clustering. We first calculate the log of the difference between a stock's price at consecutive time steps, in order to normalize the differences and make them easier to compare, which is crucial for clustering algorithms. We then compute the correlation matrix of these differences between stocks, and then convert those correlations into distances using $\sqrt{2(1 - c)}$, where c is the correlation. This distance metric assigns smaller values to highly correlated stocks and greater values to weakly correlated ones. Sklearn's agglomerative clustering then begins, using average linkage to merge two clusters until six clusters remain.

We implemented a custom gym environment. The state includes the open, close, low, high, volume, and sentiment of all stocks in the cluster for the past 5 time steps, the amount of each stock the agent currently owns, and the amount of remaining funds the agent currently has. We decided to include the previous time steps to capture trends better, since understanding a stock's past behavior is important in stock trading. The action takes on continuous values in the range $[-1, 1]$, and represents the amount of shares of a stock to buy or sell. A negative value represents the percentage of currently owned shares to sell, and a positive value represents the percentage of the agent's budget to spend on buying more shares of the stock. After every step, the agent receives a reward that is the percentage change in the agent's total portfolio value from the previous time step. After each episode, the agent

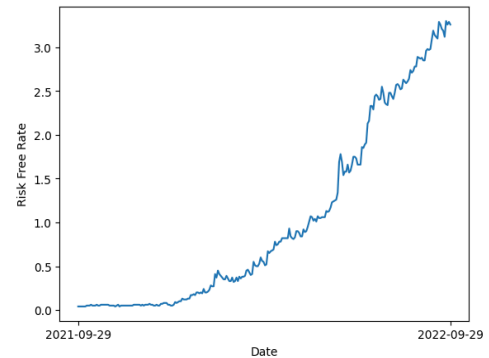


Figure 3: Daily risk-free rate over span of a year

receives an additional reward that is the Sharpe ratio, which represents how much of a profit the agent is making relative to how much it would make by taking no risk.

We experimented with different reward functions before settling on the previously mentioned choice. Our initial reward was the percentage change in the agent's total portfolio from the previous time step. As Professor Xie pointed out, this formulation does not account for risk associated with the model's actions. Subsequently, the model trained under this reward displayed erratic behavior, frequently buying and selling large amounts of shares. To incorporate risk into the reward function, we next attempted to use the Sharpe ratio as the reward function. The Sharpe ratio is given by $\frac{R_p - R_f}{\sigma_p}$, where R_p is the mean return of the portfolio, R_f is the risk-free rate, and σ_p is the standard deviation of the portfolio's returns. The risk-free rate is typically calculated using the yield of the 3-month U.S. Treasury Bill, as this is an asset with an extremely low default risk and essentially no price volatility. However, because the Sharpe ratio requires calculating the mean and standard deviation of returns, it cannot be calculated at every step. Thus, we calculated the Sharpe ratio every ten steps, but the resulting reward signal was too sparse and the model performed extremely poorly. Ultimately, we decided to mix these two approaches, creating a per-step reward of the percentage change in portfolio value and a per-episode reward of the Sharpe ratio.

4. Software

a) Code we wrote

- Preprocessing code
 - get sentiment value for each tweet
 - add sentiments from tweets to stock data
 - constrain range of dates of stock data
 - fill empty cells in risk-free rate data + constrain date range
- Custom gym environment for trading stocks
 - Implemented initialization of environment, step(), and reset() functions
 - Get observation, calculate reward, and calculate changes to state
 - Keep track of model performance during training episodes
- Evaluation Code
 - Ran trained model on testing data
 - Keep track of model performance during testing and generate graphs of performance

b) Code written by others

- gymnasium library
- StableBaseline3 for PPO
- pandas for data preprocessing
- datetime library for comparing tweet timestamps to stock timestamps
- sklearn.cluster for clustering stocks
- matplotlib for plotting results
- numpy for calculations within environment

5. Experiments and Evaluation

The stock data was split into an 80/10/10 training/validation/test split. We trained for about 1000 episodes, with each episode containing two weeks. The beginning of each episode was randomly selected from the training data.

The metric we used for evaluating our agent's performance was the difference between the model's profit and average ROI. We calculated the average ROI using the buy-and-hold strategy, which meant calculating how much profit one would make by splitting the initial budget between all stocks at the beginning of the episode and never making any other trades.

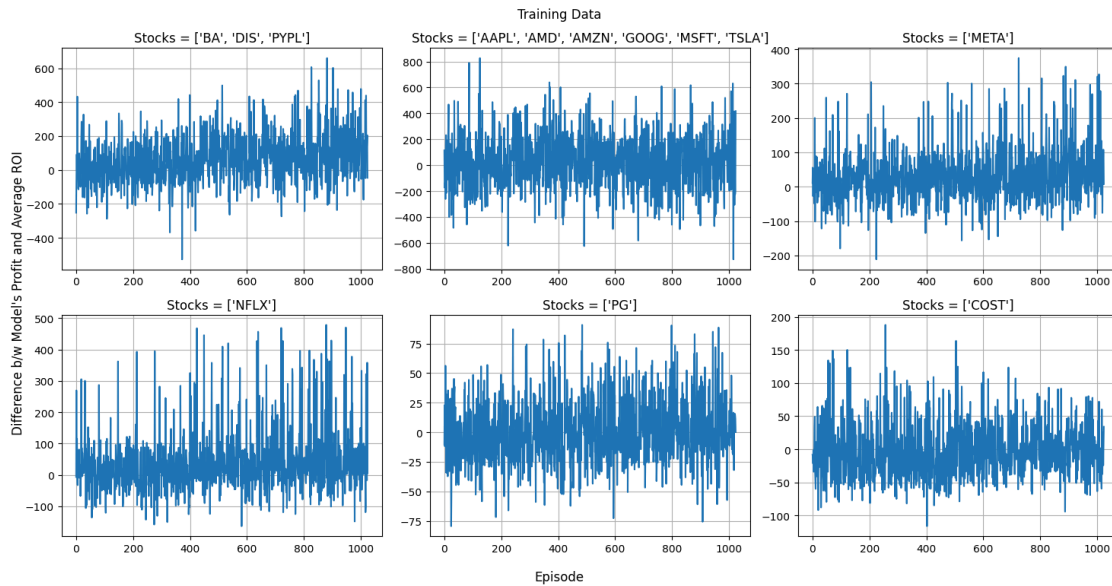


Figure 4: Difference between Model's Profit and Average ROI of each Cluster during Training

Figure 4 depicts the Model's performance during training. It is very noisy because there are 1000 episodes, but there is an upward trend in performance for all clusters, showing the model's improvement over time during training.



Figure 5: Difference between Model's Profit and Average ROI for all clusters combined during Training

Figure 5 combines the performance of all clusters during training. The upward trend in the model's performance is more clearly apparent here, as it goes from doing about the same as average ROI on average to making about \$300 more on average by the end of training.

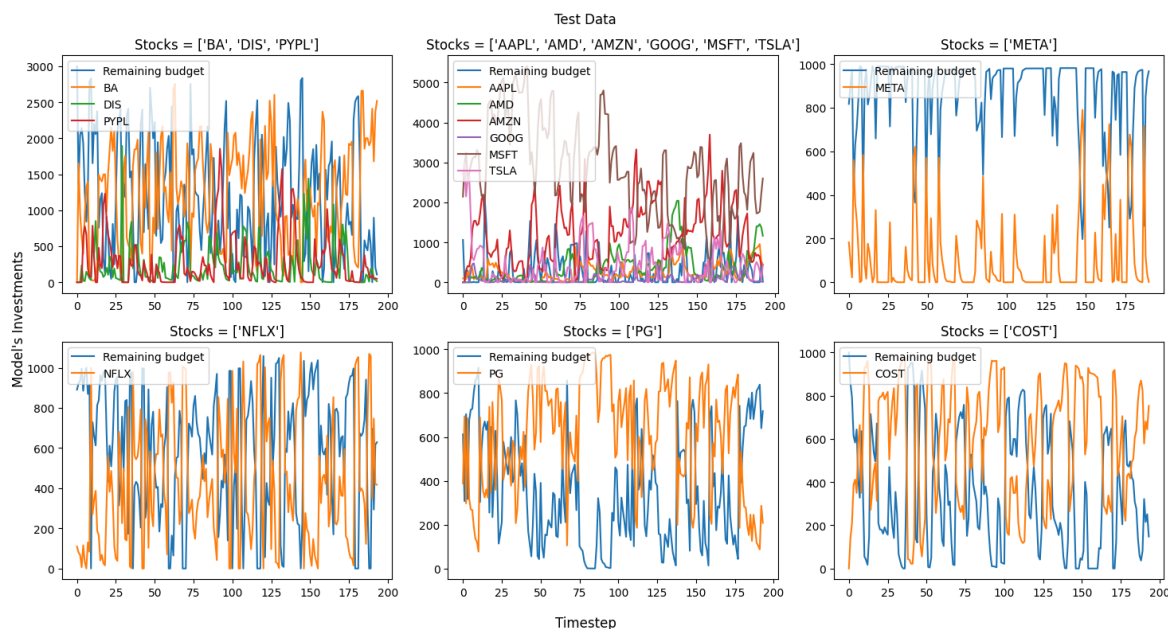


Figure 6: Model's Investments for each Cluster on the Test Data

Figure 6 depicts the decisions the model made on the test data. For the big clusters, it seems to focus more on certain stocks. It seems relatively conservative with clusters that only contain one stock, as it tends to keep more of its worth in the stock than in the remaining budget at almost all times for PG, META, and COST. This isn't the case for NFLX, likely because the Netflix stock declined quite a bit in the year 2022. The model also seems to prefer holding onto its remaining budget for the first cluster.

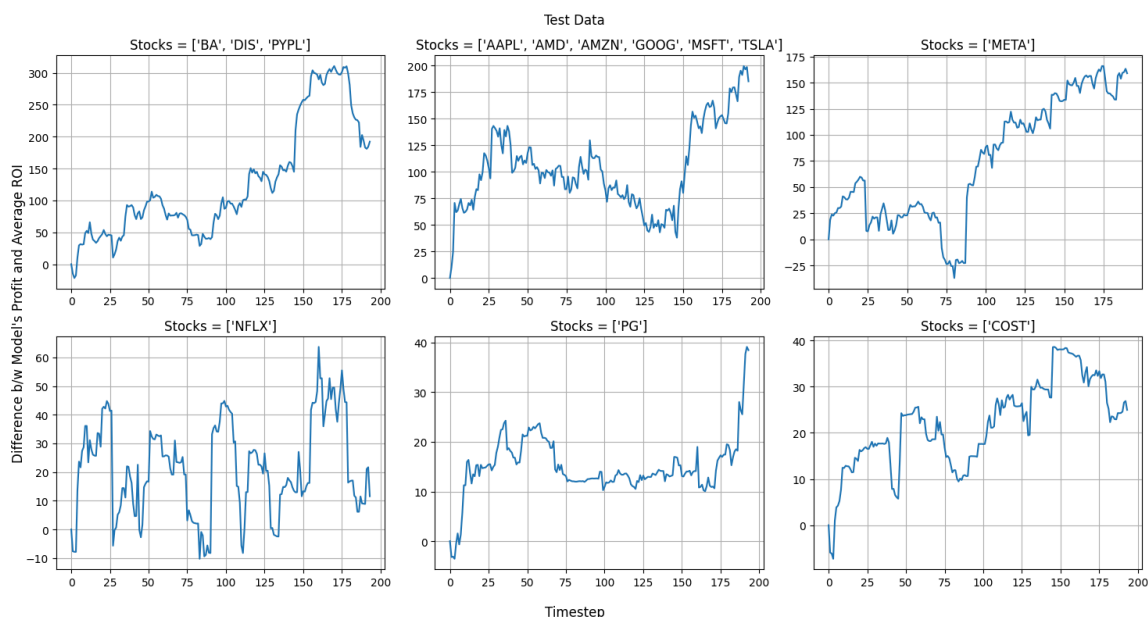


Figure 7: Difference between Model's Profit and Average ROI on Test Data

Figure 7 depicts the model's performance on each stock on the test data. All of them do well consistently, except for NFLX, which still manages to do better than the average ROI at the end.

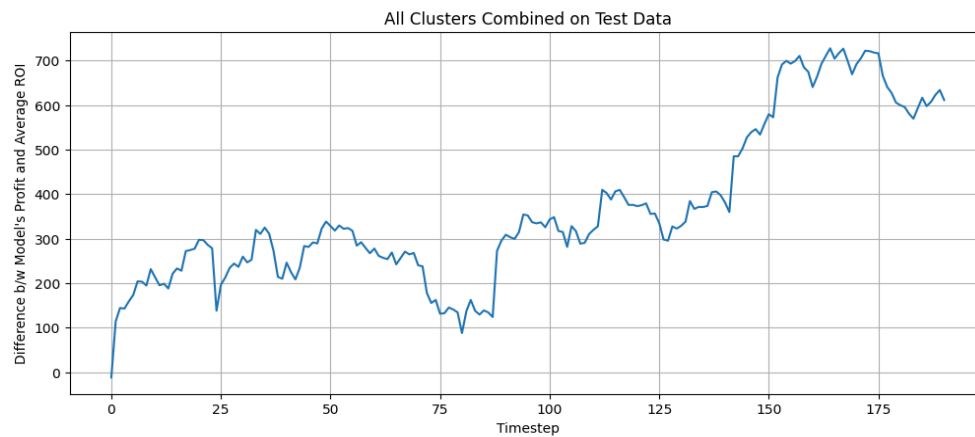


Figure 8: Difference between Model's Profit and Average ROI on Test Data for all clusters combined

As shown in Figure 8, the overall performance on all clusters combined is good, making \$600 more than the average ROI.

6. Discussion and Conclusion

One thing that was surprising is that many stocks ended up having their own cluster, showing that many stocks are actually very unique and don't necessarily follow other stocks. There are also a surprisingly large number of stocks in one of the clusters, with this cluster having AAPL, AMD, AMZN, GOOG, MSFT, and TSLA. These six stocks are some of the largest tech companies in the U.S., so it makes sense that market conditions that affect one of these companies might affect the other companies similarly.

We learned the importance of controlling the action and reward spaces. Our original action space was infinite and made it extremely easy for the model to make illegal moves, which would only slow the agent's training. Additionally, as stated in previous sections, we modified our reward function to account for risk. It's important to fully think out your action space and reward function, ensuring that you choose the options that have the least disadvantages. Failing to do so will only result in wasted effort and time, as you may fully code a flawed action/reward space that causes the model to perform poorly.

We would invest in obtaining a larger sentiment dataset. We wanted to find a dataset of tweets that included the number of likes, replies, and retweets of each tweet, as we believed these could provide insight on how popular certain sentiments were. Unfortunately, we were unable to find a dataset that contained both these statistics and enough stocks to perform our clustering methodology. Furthermore, obtaining the tweet data ourselves requires use of the Twitter API, which is quite pricey. Additionally, we would also try other RL algorithms, particularly DQN, as this model also performed well at stock trading, as shown in *Reinforcement Learning Framework for Quantitative Trading* (Gill and Yasin, 2024).

7. Individual Contribution

Shailey Joseph: I did all of the data preprocessing for running sentiment analysis on the tweets, cutting the time period of stock data, combining the sentiment analysis with the stock data, and cleaning the data of the risk free rates for calculating the sharpe ratio. I also partially worked on creating our gym environment. We jointly worked on creating the presentations and this report.

Eric Cao: I created and ran the clustering algorithm to cluster stocks with similar price trajectories. I also partially worked on creating the gym environment, which included tracking the model's training progress. I worked on the evaluation of the model and creation of the graphs displaying the model's performance. We jointly worked on creating the proposal and final presentations and reports.

8. Appendix

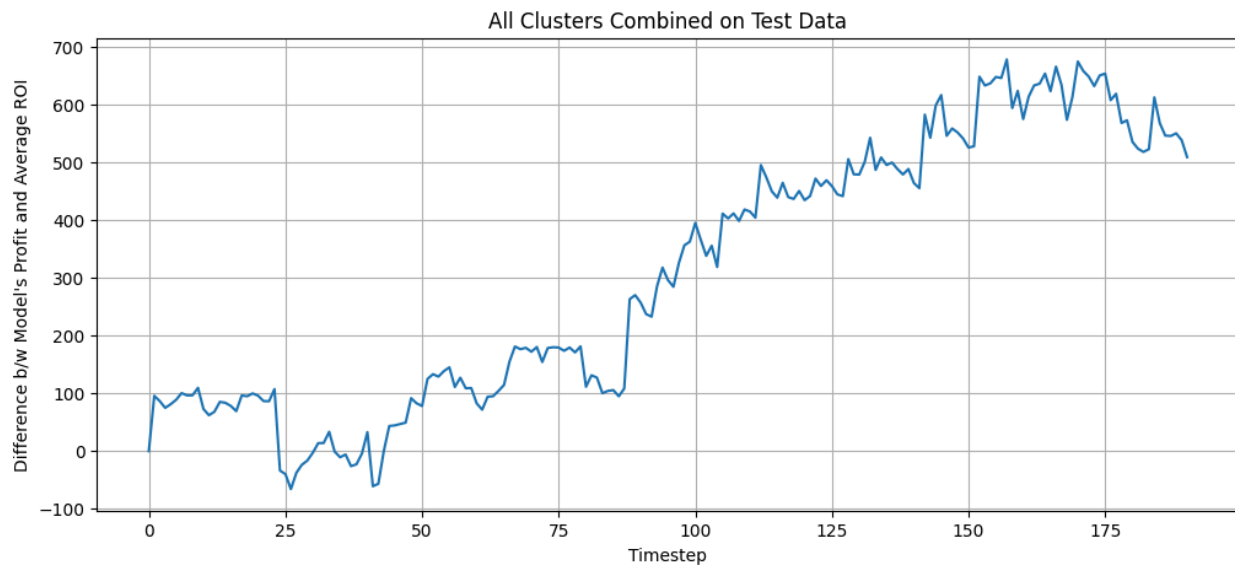


Figure 9: All Stocks combined Performance on Test Data with Original Reward Function

Figure 9 shows the overall performance of all clusters when each agent is trained with the original reward function, which doesn't incorporate the Sharpe ratio. As a result, the policy learned by the models does not account for the risk of its actions. This performance is slightly worse than the model trained with the reward function that incorporates risk, which is shown in Figure 8.

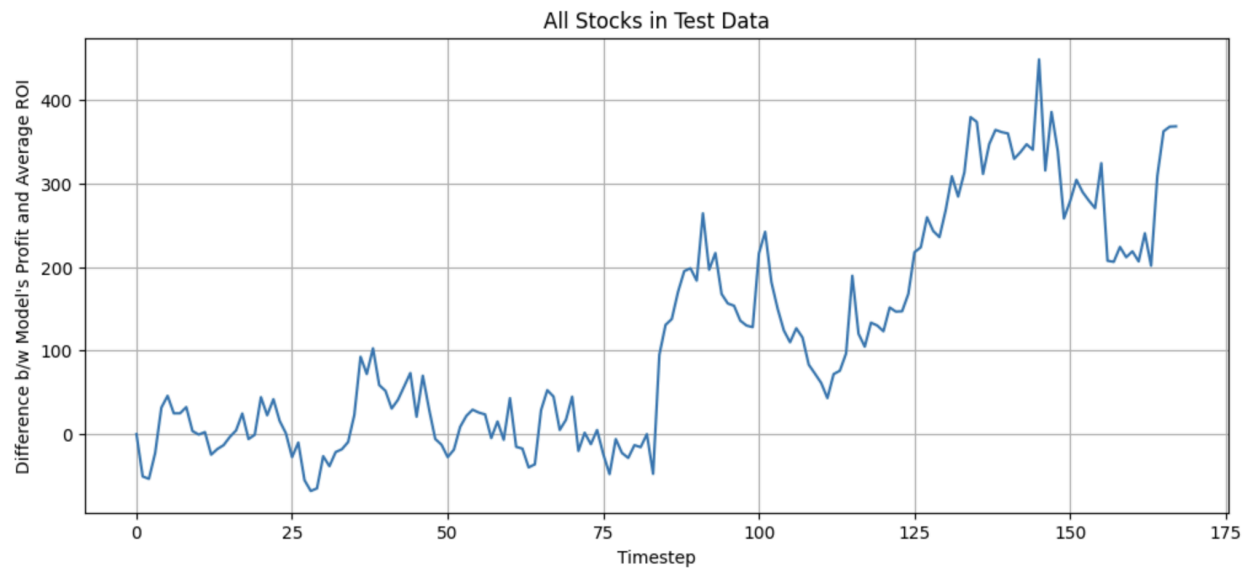


Figure 10: Test Performance of one model trained on all stocks (no clustering)

Figure 10 captures the performance of a PPO model trained on all stocks in our dataset. In other words, this agent doesn't make use of clusters. This agent performs worse than the agent made up of models that each focus on one cluster.